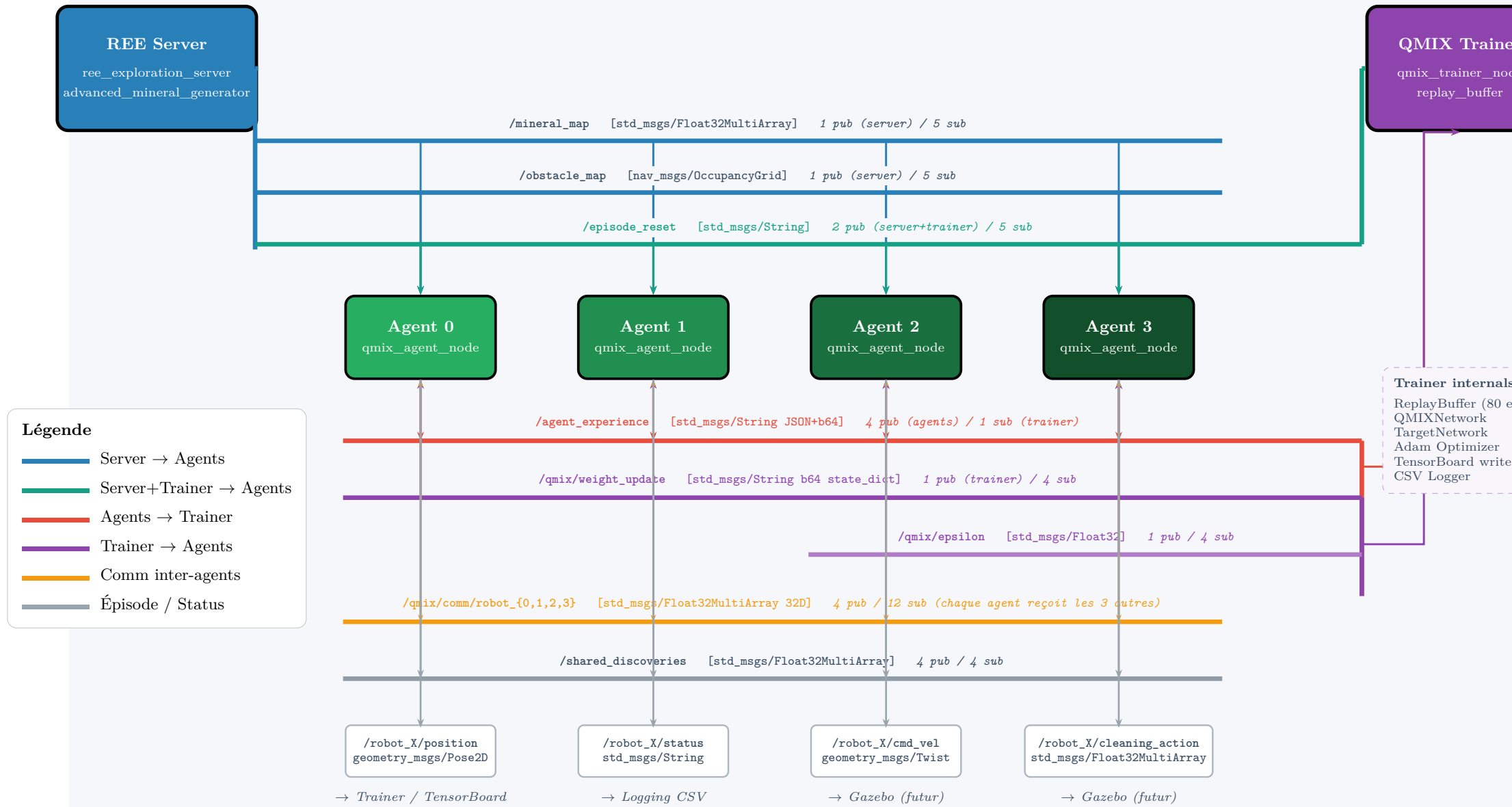




Contribution 1 — Multi-Scale CNN Encoder

local_map
(B, 6, 20, 20)
vue locale 1:1

local

CNNEncoder Local
Conv×3 + BN + ReLU
(B, 6, 20) → (B, 64, 5, 5)

Reshape
(B, 64, 5, 5) → (B, 25, 64)
25 tokens spat

regional_map
(B, 6, 20, 20)
zone 60×60 compressée

regional

CNNEncoder Régional
Conv×3 + BN + ReLU
(B, 6, 20) → (B, 64, 5, 5)

Reshape
(B, 64, 5, 5) → (B, 25, 64)
25 tokens spatiaux

Cross-Scale Attention
(4 têtes)
 $Q = \text{local_seq}$
 $K = V = \text{regional_seq}$
attended + résidu
→ (B, 25, 64)

Avg Pool
(B, 25, 64) → (B, 64)
×2 échelles

Fusion FC
concat(B, 128)
→ Linear → (B, 64)

messages reçus
(B, 3, 32)
3 autres robots

position (x,y)
(B, 2)
normalisée [0,1]

Contribution 2 — CommModule

CommModule
encode: Linear(64 → 32)
MHA(Q=own, K=V=others)
Gate:
Sigmoid(96 → 1) = $g \in [0, 1]$
out = feat + $g \cdot \text{proj}$
→ (B, 64)

Pos Encoding
Linear(2 → 16) + ReLU
→ (B, 16)

Concat
feat(B, 64) + pos(B, 16)
→ (B, 80)

Q-Network Head
FC(80 → 64) + ReLU
FC(64 → 64) + ReLU
FC(64 → 8)

argmax → ACTION
1 parmi 8 directions
N/S/E/W/NE/NW/SE/SW

INPUTS

FUSION

DÉCISION

Explication — QMIX, Contribution 1 & Contribution 2

Stage M2 — PRISMA Lab 2026 / Apprentissage par renforcement multi-agent pour l'exploration REE

Partie 0 — Le problème à résoudre

Imagine une grande carte **100×100 cases**. Des gisements de minéraux rares (REE) sont cachés un peu partout. Tu as **4 robots** pour les trouver. Chaque robot ne voit qu'une *petite zone* autour de lui. S'ils n'apprennent pas à coopérer, deux robots peuvent fouiller la même zone pendant que l'autre moitié de la carte reste inexplorée.

Objectif : apprendre aux robots à coopérer *sans programmer chaque décision à la main*. $\Rightarrow$ Solution : apprentissage par renforcement multi-agent QMIX.

Partie 1 — QMIX expliqué simplement

Apprentissage par renforcement (base)

Un robot apprend comme un enfant avec des points : essaie une action $\rightarrow$ reçoit une récompense $\rightarrow$ mémorise ce qui marche. Le réseau de neurones apprend un **score Q** par action :

$$Q(s, a) = \text{récompense espérée si on fait l'action } a \text{ depuis l'état } s$$

L'action choisie = celle avec le score Q le plus élevé.

Le problème avec plusieurs agents

Si 4 robots apprennent séparément, ils peuvent se gêner sans le savoir (2 robots qui couvrent la même zone, 2 zones jamais visitées). Il faut optimiser une *récompense globale* d'équipe.

L'idée centrale de QMIX

« Si l'équipe gagne, chaque membre a bien joué. »

QMIX apprend une valeur globale Q_{tot} et **garantit** :

$$\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0 \quad \forall \text{robot } i$$

Augmenter le score individuel d'un robot augmente toujours le score total. C'est la propriété **IGM** (Individual-Global Max).

Architecture QMIX (CTDE)

Entraînement	centralisé (accès à tout)
Exécution	décentralisée (chaque robot seul)

- Chaque robot calcule ses Q-valeurs locales Q_i
- Le **Mixing Network** combine les Q_i en Q_{tot} en utilisant l'état global de la carte
- Les poids du Mixing Network sont toujours ≥ 0 (via `torch.abs`), garantissant la monotonie IGM

Ce que chaque robot voit — observation **20×20×6**

Canal	Contenu
1	Concentration REE_Oxides
2	Concentration REE_Silicates
3	Concentration REE_Phosphates
4	Concentration REE_Carbonates

Partie 2 — Contribution 1 : Multi-Scale CNN Encoder

Le problème

Avec une observation 20×20 , le robot voit les détails immédiats mais est *aveugle* aux structures plus lointaines. Un gisement à 30 cases peut passer inaperçu.

La solution : deux zooms simultanés

- **Observation locale** : fenêtre 20×20 à résolution 1:1 $\rightarrow$ détails fins autour du robot
- **Observation régionale** : fenêtre 60×60 compressée à 20×20 (average pooling $\times 3$) $\rightarrow$ panorama large, moins précis

Chaque vue passe dans son propre **CNN** (3 convolutions + BatchNorm) et produit **25 tokens spatiaux** de 64 dimensions chacun.

Cross-Scale Attention (la fusion intelligente)

Le robot ne concatène pas bêtement les deux vues. Il utilise l'attention pour demander : « *quelle partie de la vue régionale est utile pour chaque position locale ?* »

Q = tokens locaux, $K = V$ = tokens régionaux

attended = MultiheadAttention(Q, K, V)

Connexion résiduelle + LayerNorm $\rightarrow$ stabilité de l'entraînement.

Résultat

local (B,25,64) + régional (B,25,64)

$\downarrow$ average pooling + concat + FC

h_i (B, 64) — représentation riche multi-échelle

L'agent apprend à détecter une veine de minerais $3 \times$ plus loin et à orienter ses décisions locales en conséquence.

Partie 3 — Contribution 2 : GeoCommQMIX (Communication)

Le problème

Même avec la double vue, le robot ne sait pas ce que font les 3 autres. Résultat possible : 3 robots dans la même zone, une moitié de carte jamais visitée.

Principe : un message compact de 32 nombres

À chaque pas, chaque robot encode son état interne h_i (64 dims) en un **message** m_i (32 dims) et le diffuse aux 3 autres via ROS2.

4 étapes du CommModule

1. **Encoder** : $h_i \xrightarrow{\text{FC+ReLU}} m_i$ (32) *compresse l'essentiel*
2. **Attention** : le robot lit les 3 messages reçus et pondère ceux qui lui sont les plus utiles

$$Q = m_i, \quad K = V = \{m_j\}_{j \neq i}$$

3. **Porte g** : décide *combien* écouter les voisins

$$g = \sigma(W \cdot [h_i \parallel \text{attended}]) \in [0, 1]$$

$g \approx 0$: voisins inutiles $\rightarrow$ ignorés. $g \approx 1$: voisin a trouvé quelque chose $\rightarrow$ écouté.

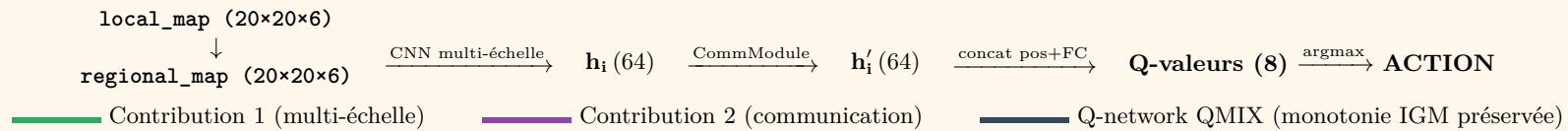
4. **Fusion résiduelle** : $h'_i = h_i + g \cdot \text{FC}(\text{attended})$

Transport physique (ROS2 DDS)

Robot i publie	/qmix/comm/robot_i
Robot i souscrit	/qmix/comm/robot_j $\forall j \neq i$

Chaque message = **Float32MultiArray** de 32 flottants. Les messages manquants (retard réseau) sont remplacés par des zéros $\rightarrow$ la porte g s'adapte automatiquement.

Partie 4 — Intégration dans QMIX : flux complet d'une décision



Entraînement (CTDE, centralisé) : le trainer recalcule toute la communication en un seul forward pass avec accès parfait aux 4 agents simultanément. Signal d'apprentissage propre et cohérent.

Exécution (décentralisée) : chaque robot utilise uniquement sa propre observation + les messages ROS2 reçus des voisins. Aucune dépendance à un serveur central. Robuste aux pannes réseau.